Hangman Game: Arquitectura y Lecciones Técnicas

Explicación de diseño modular, flujo interactivo y decisiones tomadas

UTEC • Luis Dornelles - César Arvallo

ARQUITECTURA DEL SISTEMA

Separación entre Interfaz, Estado y Utilidades

main.py

- **Presentación en Consola:** Gestiona la parte visual del juego y captura de las entradas del usuario.
- **Control de Bucle:** Organiza las fases del juego y evalúa condiciones de victoria o derrota.

game.py

- **Clase HangmanGame:** Encapsula palabra secreta, letras intentadas y contador de fallos.
- **Funciones:** Actualizan aciertos e incrementa penalizaciones en los no aciertos. También controla que los datos ingresados sean los permitidos por el juego

utils.py

- **Datos del juego:** Carga archivos de palabras para el juego.
- **import OS:** Función para limpieza de pantalla en consola.

DECISIONES TÉCNICAS

Encapsulamiento de Estado y Validación Robusta

Modelo Orientado a Objetos

- **Encapsulamiento Limpio:** La clase HangmanGame aísla el estado mutable de la partida.
- **Gestión Centralizada:** Mantiene aciertos, fallos y palabra sin dispersar variables.

Validación y Normalización

- **Filtro Estricto:** Uso de isalpha() y lower() para admitir solo caracteres válidos.
- **Manejo de Repeticiones:** Las letras repetidas se notifican sin descontar vidas.

Caracteres Especiales

- **Detección Automática:** Revela espacios y signos directamente en la categoría películas.
- **Experiencia de Juego:** Evita que los espacios aparezcan como guiones ocultos.

EXPERIENCIA DE DESARROLLO

Testimonios y Desafíos Reales del Equipo

Experiencia de Luis

- **Letras Erróneas Repetidas:** guessed_letters solo guardaba aciertos; costó evitar penalizar doble una letra errónea.
- **Curva de Aprendizaje POO:** Costó razonar cómo self conserva el estado en process_guess y cómo cada método lo muta.
- **Detalles de Depuración:** Dificultad con nombres (max_errors), persistencia de archivos y caracteres especiales (:).

Experiencia de César

- **Carga de Razonamiento:** Demandó unas 15 horas en 3 días; el reto principal fue estructurar los bucles y métodos.
- **Prolijidad y Terminal:** Uso intensivo de limpiar_pantalla(); queda para la v2.0 modularizar main.py en funciones.
- **Aporte Extra al Juego:** Incorporación de selección de categorías (A, B y C) con archivos y dificultad dinámica.

DEMOSTRACIÓN EN VIVO (1.5 MIN)

Paso a Paso del Flujo en Consola en Tiempo Real

1. Selección de Categoría

- **Validación de Menú:** Ingreso erróneo para demostrar mensaje de reintento.
- **Carga Dinámica:** Selección válida que carga el archivo de palabras correspondiente.

2. Gestión de Turnos

- **Filtro de Letra Repetida:** Comprobar que una letra ya jugada no descuenta vidas.
- **Aciertos y Fallos:** Verificación del dibujo de progreso y conteo de errores.

3. Desenlace de Partida

- **Condición de Fin:** Demostración de victoria completa o límite de 6 errores.
- **Revelación Final:** Muestra de la palabra secreta antes de salir limpiamente.

Espacio de Preguntas y Respuestas

Muchas gracias!.